— très — simplificatrice d'un choix aléatoire de la part de l'adversaire...). L'idée est alors d'estimer $p^*(\mathtt{e})$ par la méthode de Monte-Carlo. Pour cela, à partir de l'état e, on génère aléatoirement des parties jusqu'à leurs termes et on compte le rapport $p(\mathtt{e})$ entre le nombre de parties gagnées et le nombre de parties jouées. $p(\mathtt{e})$ constitue alors une estimation de $p^*(\mathtt{e})$ et on prendra $\mathtt{éval}_0(\mathtt{e}) = 2p(\mathtt{e}) - 1$.

Cette approche a permis d'améliorer sensiblement la performance de ce type de jeux.

*****

5 Sujet du TP (en 2025-2026)

L'objectif de ce TP est la réalisation d'un jeu à deux joueurs à somme nulle et information complète : le jeu intitulé *Breakthrough*.

5.1 Règles du jeu

Pour programmer un tel jeu, il faut en comprendre les règles. Les règles de *Breakthrough* sont disponibles à la page `https://brainking.com/fr/GameRules?tp=84`.

5.2 Déroulement des TP

Les TP se feront de façon individuelle (en monôme).

Le rendu du TP est :

— Un rapport en format pdf expliquant la réalisation des étapes du projet (description, résultat des tests, etc.), en les numérotant comme dans la section suivante (section 5.3) ;

— Le code du programme avec un fichier textuel expliquant comment l'installer et comment l'exécuter.

Des outils de détection de plagiat pour le fichier pdf et le code seront utilisés, à la fois pour détecter des plagiats éventuels entre groupes ou des plagiats d'autres fichiers.

Le langage de programmation sera Java.

5.3 Étapes de la réalisation

1. Définir la classe des états du jeu (temps estimé : 90 minutes) : Vous devrez définir une classe pour représenter un état du jeu, avec la possibilité d'afficher un tel état (de façon simple, mais en numérotant les lignes et les colonnes du plateau [11]) et de modifier une case du jeu. On définira les constantes, instances de cette classe correspondant à l'état initial et à l'état d'échec (qui correspond à la réalisation d'une action illicite).

2. Définir la classe des joueurs (temps estimé : 10 minutes) : Cette classe permettra notamment de définir les deux instances de cette classe : `joueurBlanc` (qui commence, à ce jeu) et `joueurNoir`.

3. Calcul d'un successeur d'un état (temps estimé : 90 minutes) : Une action est donnée par un joueur j, un état e et un triplet (ℓ, c, d) où ℓ est un numéro de ligne, c un numéro de colonne et $d \in \{-1, 0, 1\}$. La réalisation de cette action revient pour j à sélectionner sa pièce située à la ligne ℓ et à la colonne c et à la déplacer d'une case dans la bonne direction, avec un décalage de colonne de d. Si l'action (ℓ, c, d) est possible pour le joueur j dans l'état e alors le résultat de cette action sera un état s qui est une copie de e sauf que : (1) la case (ℓ, c) est vide, (2) il y a un pion de la couleur de j dans la case $(\ell + a, c + d)$ où a vaut -1 si $j =$ `joueurNoir` et 1 sinon. Bien entendu, l'action n'est possible que sous certaines conditions (case de départ contenant bien un pion du joueur j ;

11. Sur la description du jeu en ligne, les colonnes sont nommées par des lettres, de A à H. Vous leur donnerez plutôt des numéros de 0 à 7, par exemple. Il est conseillé de paramétrer le nombre de colonnes par une constante de la classe : cela permet de jouer au jeu avec 8 lignes et nc colonnes, où nc vaut 8 dans le jeu original, mais peut valoir moins ($nc = 5$, par exemple), voire plus, pour tester le jeu.

Les lignes sont numérotées sur cette page Web par des nombres 1 à 8 ; là aussi, vous pouvez les numéroter de 0 à 7. On gardera toujours 8 lignes.

case d'arrivée libre ou alors prise en diagonale d'un pion adverse ; case d'arrivée ne sortant pas du plateau). Si l'action n'est pas réalisable, alors s sera l'état d'échec.

4. Calcul de l'ensemble des successeurs d'un état (temps estimé : 15 minutes) : Étant donné un joueur et un état, l'objectif est de calculer l'ensemble des successeurs de cet état pour ce joueur (l'ensemble des états obtenus en appliquant toutes les actions possibles par ce joueur sur cet état).

5. Jeu à deux joueurs humains (temps estimé : 90 minutes) : Programmez le jeu pour deux joueurs humains. Si l'état est final, le jeu doit indiquer qui est gagnant. Sinon, le joueur courant doit indiquer l'action par le triplet (ℓ, c, d) (on vérifiera que cette action est licite et on refusera dans le cas où elle ne l'est pas). Testez ce programme.

6. Programmez l'algorithme minimax (temps estimé : 120 minutes) : En utilisant une fonction $\texttt{éval}_0$ naïve, programmez et testez l'algorithme minimax. On demande plus précisément :

- Une implantation de l'algorithme vu en cours, sur ce jeu ;
- Une interface pour qu'un être humain puisse jouer à ce jeu ;
- La possibilité de voir deux joueurs artificiels jouer l'un contre l'autre ;
- Une estimation du temps de calcul en fonction de la profondeur c (nombre de coups à regarder à l'avance), pour $c \in \{1, 2, 3\}$ (ou au-delà).

On pourra, pour cette implantation, prendre la fonction $\texttt{éval}_0$ qui à un joueur j et un état associe $n_j - n_{\bar{j}}$ où n_j (resp. $n_{\bar{j}}$) est le nombre de pions de la couleur de n_j (resp. de la couleur de l'autre joueur).

7. Programmez l'algorithme minimax avec élagage $\alpha\beta$ (temps estimé : 60 minutes) : Implantez l'élagage $\alpha\beta$ et refaites les tests de temps de calcul.

8. Comparaison automatique de deux fonctions $\texttt{éval}_0$ (temps estimé : 60 minutes) : Étant donné deux fonctions $\texttt{éval}_0$, notées $\texttt{éval}_0^a$ et $\texttt{éval}_0^b$ dans la suite, on se demande quelle est la meilleure des deux en termes de jeu. Pour cela, l'intuition est de faire jouer un joueur automatique muni de $\texttt{éval}_0^a$ et un joueur automatique muni de $\texttt{éval}_0^b$ pour voir celui qui gagne. À noter :

- Pour que $\texttt{éval}_0^a$ soit meilleure que $\texttt{éval}_0^b$, il faut que $\texttt{éval}_0$ « gagne » dans les deux parties : celle où $\texttt{éval}_0^a$ commence et l'autre (ou, au moins, qu'elle gagne une des parties et qu'elle soit à égalité pour l'autre partie).
- Avec ce critère, on peut ne pas arriver à différencier les deux fonctions, dans ce cas, on reprendra la comparaison à une profondeur différente (on commencera à $c = 1$, puis on passera à $c = 2$, $c = 3$, etc., en restant à une profondeur garantissant un temps de calcul « raisonnable »).

9. Implantez une méthode pour trouver une bonne fonction $\texttt{éval}_0$ (temps estimé : 60 minutes) : La fonction $\texttt{éval}_0$ peut être paramétrée par un n-uplet de réels x : on la note $\texttt{éval}_0^x$. L'objectif de cette dernière partie du TP est de trouver une « bonne » fonction $\texttt{éval}_0$ en appliquant une des méthodes ci-dessous :

1. Générez aléatoirement p valeurs différentes pour le n-uplet x puis organisez (de façon automatique) une compétition entre valeurs de x, afin de maximiser $\texttt{éval}_0$. On pourra prendre $p = 8$ et il est conseillé de prendre une puissance de 2 pour p pour organiser la compétition.
2. Partir d'un paramètre x initial et obtenir par une « petite » variation x' et comparez $\texttt{éval}_0^x$ avec $\texttt{éval}_0^{x'}$. Repartez du gagnant et recommencez. Le « pas » utilisé est un majorant de la distance entre x et x' (x sera choisi dans la boule centrée sur x et de rayon ce pas, où la boule est définie par la distance choisie entre n-uplets de réels). Il faut diminuer le pas progressivement.
3. Application de la méthode de Monte-Carlo (voir cours).

Remarque 2 La méthode 1 est plus facile que la méthode 2 qui est plus facile que la méthode 3. La notation du TP tiendra compte de la facilité de la méthode choisie.